

第3章 流程控制

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

本章学习目标

- 掌握 if/else、switch、三种循环与 C 的异同,特别是条件类型与悬挂 else
- 掌握范围 for 循环与结构化绑定,理解它的底层是迭代器
- 理解迭代器失效的四种场景,掌握 erase-remove 惯用法
- 掌握 break/continue/goto 的语义与限制,理解 [[fallthrough]] 属性
- 掌握 C++17 的 if/switch 初始化语句,写出作用域更小的代码

从 C 到 C++:本章主题如何迁移

流程控制是 C 工程师最熟悉的地盘:if、for、while、switch、goto,语法几乎原样保留。C++ 的改动集中在三处:新增范围 for 让遍历容器的代码短了一半;新增 if/switch 初始化语句让"临时变量只在分支内可见";以及容器时代带来的新坑——迭代器失效,这是裸数组时代根本不存在的概念。另外,switch 的 case 穿透在 C++ 里有了官方标注方式 [[fallthrough]],代码评审不再靠猜。

学完本章,你的循环代码应该默认写成"范围 for + const auto&"或"迭代器 + 下标"两种风格,并且能清楚地解释为什么不能在遍历时往 vector 里插入元素。

前置知识自查

- C 的 if/else、switch、for/while/do-while 与 break/continue/goto
- C 的悬挂 else 规则与 switch 穿透行为
- 第1章的 std::vector 与 std::map 基本操作
- 第2章的运算符优先级与短路求值

本章学习路线

1. 第一阶段(熟悉):§3.1-§3.2 条件与分支,重点理解条件类型要求与 case 穿透
2. 第二阶段(重点):§3.3-§3.4 三种循环与范围 for,把遍历习惯改过来
3. 第三阶段(难点):§3.5 迭代器失效,这是容器时代的必修课
4. 验收标准:能解释 vector 扩容为何使迭代器失效,能写出 erase-remove 惯用法

本章知识导图

- §3.1 if/else:条件类型、悬挂 else、if 初始化语句
- §3.2 switch:case 穿透、[[fallthrough]]、switch 初始化语句
- §3.3 三种经典循环:for/while/do-while
- §3.4 范围 for:底层迭代器、const auto&、结构化绑定
- §3.5 迭代器失效:扩容、删除、erase-remove 惯用法
- §3.6 break/continue/goto 与 [[fallthrough]]

§3.1 if/else:条件与分支

if/else 的语法与 C 完全一致,包括悬挂 else 规则:else 与最近的未配对 if 结合。看代码时先数花括号再读逻辑,否则 else 会配对到想不到的 if 上。与 C 相同,C++ 的条件上下文接受数值与指针(非零即真),与 Go、Rust 要求条件必须是 bool 不同——这也是 C 家族容易写出 if (x = 1) 的原因:赋值表达式的值就是被赋的值,恒为真,编译器通常只给警告。生产规范:单行 if 也要写花括号;条件里禁止赋值;判定"相等"永远写 x == 1,并把常量放左边(1 == x)可让误写赋值直接编译失败。

C 的写法	C++ 的写法	差异
条件允许数值/指针	条件允许数值/指针	相同;Go/Rust 必须 bool
悬挂 else 就近配对	悬挂 else 就近配对	完全相同
无	if (init; cond) 初始化语句	C++17 新增,变量限分支内
if (x = 1) 是赋值	if (x = 1) 是赋值	相同;想写比较先写 x == 1

C++17 的 if 初始化语句是生产中常用的小改进:if (auto it = m.find(k); it != m.end()) 把查找与判断写在一起,变量作用域限定在分支内,不会泄漏到函数体。注意初始化语句与条件之间用分号分隔,条件可以是任意表达式。同样的写法也适用于 while 循环:while (auto ch = getchar(); ch != EOF) 在每次迭代重新求值条件。习惯这个写法后,你会发现大量"临时变量+判断"的两行代码能压缩成一行,且不会忘记清理。

示例 3-1 条件与 if 初始化语句

```

#include <iostream>
#include <string>

int main() {
    int score = 85;
    if (score >= 90) {
        std::cout << "优秀\n";
    } else if (score >= 60) {
        std::cout << "及格\n";        // 悬挂 else 与 C 相同:就近配对
    } else {
        std::cout << "不及格\n";
    }

    // C++17:if 初始化语句,变量只活在分支里
    if (int pass = score - 60; pass >= 0) {
        std::cout << "超出及格线 " << pass << " 分\n";
    }
    // 此处 pass 不可见

    // 指针直接作条件:非零即真(与 C 相同)
    std::string* p = nullptr;
    if (p) {
        std::cout << "非空\n";
    } else {
        std::cout << "空指针\n";
    }
    return 0;
}

```

§3.2 switch:case 穿透与 fallthrough

switch 的规则与 C 相同:case 标签必须是整型常量表达式(枚举可以),执行到 break 才退出,否则向下穿透。C++ 与 C 的三点差异:第一,switch 内不能跨过带初始化的变量声明——case 2: 里 int x = 0; 而 case 1: 跳到 case 2 会越过初始化,是编译错误,解决办法是给 case 加花括号;第二,C++17 起可用 [[fallthrough]] 显式标注"故意穿透",替代 C 时代的裸注释,配合 -Weimplicit-fallthrough 编译器还能帮你抓漏写的 break;第三,C++17 起 switch 同样支持初始化语句。

工程习惯:case 少于 5 个时 switch 清晰可读,再多建议换查找表(数组或 std::map 按值取结果)——第4章讲函数指针后会给出具体写法。每个 switch 都写 default 分支,枚举的 switch 尤其要处理将来新增枚举值的情况,编译器配合 -Wswitch 可以警告漏掉的枚举分支。

C 的写法	C++ 的写法	差异
case 穿透,裸注释	case 穿透,[[fallthrough]] 标注	C++17 属性,编译器可检查
case 里可声明变量	case 里声明变量需加花括号	C++ 更严格:禁止跨初始化跳转
无	switch (init; 表达式)	C++17 新增

示例 3-2 switch 与 [[fallthrough]]

```

#include <iostream>

const char* weekday(int day) {
    switch (day) {                // case 必须是整型常量表达式
        case 1: return "周一";
        case 2: return "周二";
        case 3: return "周三";
        case 4: return "周四";
        case 5: return "周五";
        case 6:
        case 7: return "周末";    // 隐式穿透:case 6 落入 case 7
        default: return "非法";
    }
}

int main() {
    // 穿透演示:6 和 7 都得到"周末"
    for (int d = 1; d <= 7; ++d) {
        std::cout << d << ":" << weekday(d) << " ";
    }
    std::cout << "\n";

    // [[fallthrough]]:显式声明故意穿透(C++17)
    int x = 2;
    switch (x) {
        case 1:
            std::cout << "一";
            [[fallthrough]];      // 告诉编译器与读者:不是忘了 break
        case 2:
            std::cout << "二\n";
            break;
        default:
            std::cout << "其他\n";
    }

    // case 内声明变量必须加花括号,否则编译错误(注释展示)
    // switch (x) {
    //     case 1: int v = 0; break;    // 错误:可跨过初始化
    //     case 2: break;
    // }
    return 0;
}

```

⚠ 易错点 3:do-while 的分号与 while 条件副作用

do-while 的 while (条件); 结尾分号不能丢——丢了分号,后面的语句会被当成循环体的一部分,行为完全错乱。另一个高频错误:在 while 条件里写带副作用的表达式,如 while (getline(cin, line)) 没问题,但 while (i++ < n && ...) 这类写法在维护时极易出错,条件里的变量每次迭代都在变,调试困难。生产规范:条件里只做判断,副作用放到循环体内显式写。

§3.3 三种经典循环

for/while/do-while 与 C 完全相同,包括初始化与迭代子句里的逗号表达式。生产习惯:循环变量用 int 还是 std::size_t 要区分——遍历容器下标建议 std::size_t(避免与 size() 的无符号比较警告),计数次数明确小时用 int;死循环统一写 for(;;),不要写 while(1);do-while 至少执行一次,适合菜单与输入重试;循环体内不变的表达式提到循环外,如 size() 在迭代中不变时可缓存,但注意缓存与"每次重新取"在容器被修改时结果不同,先想清楚再优化。

示例 3-3 三种循环与死循环

```
#include <iostream>

int main() {
    // for:与 C 完全相同的三段式
    int sum = 0;
    for (int i = 1; i <= 100; ++i) {
        sum += i;
    }
    std::cout << sum << "\n";    // 5050

    // while:先判后做
    int n = 5, cnt = 0;
    while (n > 0) { n >>= 1; ++cnt; }
    std::cout << cnt << "\n";    // 3

    // do-while:至少执行一次,菜单程序的标配
    int choice = 0;
    do {
        std::cout << "选 1-3: ";
        std::cin >> choice;
    } while (choice < 1 || choice > 3);
    std::cout << "你选了 " << choice << "\n";

    // 死循环惯用法:for(;;)
    int k = 0;
    for (;;) {
        if (++k >= 3) break;
    }
    std::cout << k << "\n";    // 3
    return 0;
}
```

§3.4 范围 for:现代遍历的正统

范围 for 是 C++11 对循环的最大改进:for (const auto& x : 容器) 在底层展开为迭代器循环,代码量与出错率同时下降。三个要点:第一,只读遍历用 const auto&,修改遍历用 auto&,按值 auto 会拷贝元素(对 std::string 这类带堆内存的类型是真实开销);第二,范围 for 内部持有迭代器,遍历中修改容器结构(插入、删除)会使迭代器失效,行为未定义;第三,遍历 std::map 时用结构化绑定 for (const auto& [k, v] : m)。什么时候不用范围 for:需要下标参与算法(如冒泡排序)、需要反向遍历、需要跳过或提前删除元素时,回退到传统循环。

示例 3-4 范围 for 与结构化绑定

```

#include <iostream>
#include <vector>
#include <map>

int main() {
    std::vector<int> v{10, 20, 30};

    // 只读遍历:const auto& 零拷贝
    for (const auto& x : v) {
        std::cout << x << " ";
    }
    std::cout << "\n";

    // 修改遍历:auto&
    for (auto& x : v) {
        x *= 2;
    }

    // 结构化绑定遍历 map(C++17)
    std::map<std::string, int> scores{{"张三", 90}, {"李四", 75}};
    for (const auto& [name, score] : scores) {
        std::cout << name << ":" << score << " ";
    }
    std::cout << "\n";

    // 范围 for 的底层:传统迭代器循环
    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " ";    // 20 40 60
    }
    std::cout << "\n";
    return 0;
}

```

§3.5 迭代器失效:C 数组没有的新坑

裸数组的下标访问在删除元素后依然“可用”(虽然语义错乱),但 `std::vector` 的迭代器在扩容或删除后直接失效——解引用失效迭代器是未定义行为。这是容器时代的新坑,C 里没有对应物。四种典型场景:vector 扩容使全部迭代器失效;vector erase 使被删位置及之后的迭代器失效;map/unordered_map 删除只使被删迭代器失效(其他仍有效,所以 map 可以安全地边遍历边删除,用 `it = m.erase(it)` 的写法);任何容器的析构使全部迭代器失效。失效迭代器与悬挂指针是同一类问题:值还在内存里,但所有权已转移,任何使用都是 UB。

⚠ 易错点 1:遍历时删除元素

`for (auto it = v.begin(); it != v.end(); ++it) { if (*it == 2) v.erase(it); }` 是错误的:erase 之后 `it` 失效,++it 是 UB。正确姿势是 erase-remove 惯用法:`v.erase(std::remove_if(v.begin(), v.end(), 条件), v.end())`——先集中移除,再一次 erase,整个过程中迭代器由算法内部安全处理。

示例 3-5 迭代器失效与 erase-remove 惯用法

```

#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    // 场景一:扩容使迭代器失效
    std::vector<int> v{1, 2, 3};
    auto it = v.begin();
    v.push_back(4);           // 可能扩容,it 已失效
    // std::cout << *it;      // 不要用:UB

    // 生产习惯:先 reserve 预分配,遍历中插入不失效
    v.reserve(100);
    it = v.begin();
    v.push_back(5);
    std::cout << *it << "\n";    // 1:容量充足,迭代器有效

    // 场景二:删除元素,用 erase-remove 惯用法
    std::vector<int> w{1, 2, 3, 4, 5};
    w.erase(std::remove_if(w.begin(), w.end(),
                           [](int x) { return x >= 3; }),
            w.end());
    for (int x : w) std::cout << x << " ";    // 1 2
    std::cout << "\n";
    return 0;
}

```

⚠ 易错点 2:范围 for 里插入元素

for (int x : v) { v.push_back(1); } 是 UB:范围 for 的底层迭代器在扩容后失效,++it 越界。若确实需要边遍历边插入,改用带下标的循环并在插入后重新获取 size(),或先收集再统一处理。

§3.6 break/continue/goto 与收尾控制

break 跳出最近一层循环或 switch;continue 跳过本次迭代进入下一次;goto 可以跳出任意深度的嵌套(但不能跳出函数),代价是代码结构被破坏,生产代码只在错误清理等少数场景使用。C++ 对 goto 有一处限制:不允许跳到带初始化变量的作用域之前——这与 switch 的 case 限制同理,都是为了防止跳过初始化。需要跳出多层循环时,优先考虑把循环抽成函数用 return 退出,可读性远好于 goto;读到别人的 goto 代码时,先确认标签位置与跳转方向,不要急着重构。

最后提醒复杂度:嵌套循环的层数要心里有数,三重循环在数据量过万时基本不可用。写循环前先估计数据规模与每层迭代次数,再把可提前退出的条件提到最外层——这是工程思维的第一课,也是第10章讲算法复杂度的铺垫。另外,循环内的分支会破坏流水线预测,热点循环里把最可能的分支写在前面、把 if 提成查表或开关,往往比微调比较写法收益更大;排查性能问题时先看循环,再谈其他。

C 的写法	C++ 的写法	差异
break/continue/goto	break/continue/goto	语义完全相同
goto 可跳任意标签	goto 不可跳过带初始化的变量	C++ 增加编译期检查
无	[[fallthrough]]、[[nodiscard]]	C++ 用属性标注意图

示例 3-6 break、continue 与 goto

```

#include <iostream>

int main() {
    // continue:跳过本次迭代
    for (int i = 1; i <= 10; ++i) {
        if (i % 2 == 0) continue;    // 跳过偶数
        std::cout << i << " ";      // 1 3 5 7 9
    }
    std::cout << "\n";

    // break:跳出最近一层循环
    int total = 0;
    for (int i = 1; ; ++i) {
        total += i;
        if (total > 20) break;
    }
    std::cout << total << "\n";      // 21

    // goto:跳出双层嵌套(生产代码慎用)
    int depth = 0;
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (i == 1 && j == 1) goto found;
            ++depth;
        }
    }
found:
    std::cout << "depth = " << depth << "\n";    // 3

    // goto 不能跳过带初始化的变量:编译错误(注释展示)
    // goto skip;
    // int bad = 0;
    // skip:
    return 0;
}

```

本章速查表

主题	速查内容
条件类型	数值/指针非零即真(与 C 相同);Go/Rust 必须 bool
悬挂 else	else 就近配对;if (x = 1) 是赋值,不是比较
if 初始化	C++17:if (auto it = m.find(k); it != m.end())
switch	case 需整型常量;忘 break 会穿透;加 [[fallthrough]] 标注
switch 变量	case 内声明变量必须加花括号
三种循环	for/while/do-while 与 C 相同;菜单用 do-while
死循环	写 for(;;),不写 while(1)
范围 for	只读 const auto&,修改 auto&,按值会拷贝
遍历 map	for (const auto& [k, v] : m) 结构化绑定
迭代器失效	扩容/erase 后全部或部分失效,解引用是 UB
删除元素	erase-remove 惯用法;遍历中不插入、不删除
break/continue	与 C 相同;continue 跳过本次迭代
goto	可跳多层嵌套;不可跳过带初始化的变量
工程规范	循环里先 reserve;范围 for 内不改容器结构

最小必会清单

- 能解释条件类型的 C/C++ 语义与 Go/Rust 的差异;能识别 if (x = 1) 的错误
- 能写出 if 初始化语句与 switch 初始化语句的正确语法
- 能说出 case 穿透的规则与 [[fallthrough]] 的用途
- 能写出三种循环,解释 do-while 与 while 的区别
- 能写出范围 for 的三种形式:const auto&、auto&、结构化绑定
- 能解释 vector 扩容为什么使迭代器失效,并给出两种规避方式
- 能徒手写出 erase-remove 惯用法(含 lambda 条件)
- 能解释 goto 的限制:不能跳过带初始化的变量

自测题

Q `int x = 0; if (x = 1) { ... }` 会进入分支吗?为什么这个写法危险?

✓ 参考答案

会进入:`x = 1` 是赋值表达式,值为 1,非零即真。危险在于本意可能是 `x == 1`,误写后行为完全不同且编译器只给警告。生产规范:条件里禁止赋值,或用 `if (int y = ...; y)` 的初始化语句写法。

Q `switch` 里 `case 1: int v = 0; break;` 为什么是编译错误?如何修复?

✅ 参考答案

case 穿透可能从其他标签跳进 case 1 的作用域,越过 v 的初始化,违背"变量必须初始化后才能使用"的规则。修复:把声明包进花括号 case 1: { int v = 0; break; }。

Q for (const auto& [name, score] : scores) 中 const auto& 的作用是什么?去掉会怎样?

✅ 参考答案

const auto& 绑定到 map 元素的引用且只读,零拷贝。去掉 const 会允许修改;写成 auto 会拷贝整个 pair,对大数据容器是性能浪费。

Q 下面的代码哪里错了?for (int x : v) { if (x == 2) v.push_back(3); }

✅ 参考答案

遍历中 push_back 可能触发扩容,范围 for 底层的迭代器随之失效,++it 是未定义行为。修复:先收集要插入的值,遍历结束后统一插入;或改用下标循环并每次重新取 size()。

Q 写出删除 vector<int> 中所有偶数元素的惯用法(要求一行内完成逻辑)。

✅ 参考答案

v.erase(std::remove_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; }), v.end()); remove_if 把满足条件的元素移到尾部并返回新逻辑结尾,erase 一次性删除,全程迭代器安全。

Q 为什么可以在遍历 std::map 时删除当前元素,却不能在遍历 std::vector 时删除当前元素?

✅ 参考答案

map 是节点型容器,删除只使被删元素的迭代器失效,前一个迭代器依然有效,所以惯用法是 it = m.erase(it);(erase 返回下一个迭代器)。vector 是连续存储,erase 会使被删位置及之后全部迭代器失效,遍历中删除必然踩空。因此 vector 的删除一律走 erase-remove 或先收集后统一处理。

章末实验题:学生成绩单菜单程序

实验 3:学生成绩单菜单程序

实验目标:写一个命令行菜单程序:录入学生成绩、统计平均分、删除不及格学生,把全章流程控制知识点全部用上。

实验要求:

- 任务 1:菜单循环用 do-while + switch,退出用 goto 跳出(覆盖:do-while、switch、default、goto)
- 任务 2:录入分支:用 if 检查成绩范围,越界用 continue 重进菜单(覆盖:if、continue)
- 任务 3:统计分支:范围 for 累加,发现极低分用 break 提前终止(覆盖:范围 for、break、setprecision)
- 任务 4:gradeOf() 用 switch 判等级,case 10/9 共享分支(覆盖:case 穿透)
- 任务 5:dropFailures() 用 erase-remove 惯用法删除不及格者(覆盖:迭代器失效的正确处理)
- 任务 6:手动验证:录入 5 人,统计,删除不及格,再统计(覆盖:完整流程)
- 任务 7:用 -Wall -Wextra 编译,零警告;每处 switch 都有 default(覆盖:工程规范)

实验环境:任意 C++17 编译器。编译命令:g++ -std=c++17 -Wall -Wextra gradebook.cpp -o gradebook

第一步 写结构体与菜单骨架(do-while + switch),编译通过再加统计

第二步 加 gradeOf 与 dropFailures,注意 case 花括号与 erase-remove 的写法

第三步 后补 goto 退出与边界验证,运行三组数据核对结果

参考答案(第一部分:数据结构)

```
// 学生成绩单菜单程序(第一部分:数据结构)
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>

struct Student {
    std::string name;    // 姓名
    double score;        // 成绩
};
```

✅ 参考答案(第二部分:判级与删除)

```
// 第二部分:判级与删除(覆盖:switch、case 穿透、erase-remove)
const char* gradeOf(double score) {
    switch (static_cast<int>(score) / 10) {    // 任务 4:switch 判等级
        case 10:
        case 9: return "优";    // case 穿透:10 与 9 共享分支
        case 8: return "良";
        case 7:
        case 6: return "及格";
        default: return "不及格";
    }
}

// 任务 5:删除不及格学生。
// 错误示范:for (auto it = list.begin(); it != list.end(); ++it)
//           if (it->score < 60) list.erase(it);    // it 失效,UB
// 正确做法:erase-remove 惯用法(lambda 详见第10章,这里当回调用)
void dropFailures(std::vector<Student>& list) {
    list.erase(std::remove_if(list.begin(), list.end(),
                               [](const Student& s) {
                                   return s.score < 60.0;
                               }),
               list.end());
}
```

✅ 参考答案(第三部分:main 菜单循环)

```
// 第三部分:main 菜单循环(覆盖:do-while、switch、break、continue、goto)
int main() {
    std::vector<Student> students;
    int choice = 0;
    do {
        // 任务 1:菜单至少显示一次
        std::cout << "1.录入 2.统计 3.退出: ";
        std::cin >> choice;
        switch (choice) {
            case 1: {
                std::string name;
                double score;
                std::cin >> name >> score;
                if (score < 0.0 || score > 100.0) { // 任务 2
                    std::cout << "成绩越界,忽略\n";
                    continue; // continue:回到菜单循环
                }
                students.push_back({name, score});
                break;
            }
            case 2: {
                if (students.empty()) { std::cout << "暂无数据\n"; break; }
                double sum = 0.0;
                int cnt = 0;
                for (const auto& s : students) { // 任务 3:范围 for
                    if (s.score < 30.0) {
                        std::cout << "发现极低分,中止统计\n";
                        break; // break:跳出范围 for
                    }
                    sum += s.score;
                    ++cnt;
                }
                std::cout << std::fixed << std::setprecision(1);
                if (cnt > 0) std::cout << "平均分: " << sum / cnt << "\n";
                std::cout << "人数: " << students.size() << "\n";
                for (const auto& s : students) { // 打印每人等级
                    std::cout << s.name << ": " << gradeOf(s.score) << "\n";
                }
                break;
            }
            case 3:
                std::cout << "再见\n";
                goto done; // goto:跳出循环与 switch
            default:
                std::cout << "无效选项\n";
                break;
        }
    } while (choice != 3);
done:
    std::cout << "程序结束\n";
    return 0;
}
```

设计说明:case 1 与 case 2 用花括号包住声明(switch 内声明变量的规则);continue 在 switch 内跳转的是外层 do-while;goto done 演示了跳出嵌套的正确场合——退出多层结构。dropFailures 放在统计前调用可演示完整流程:录入、删除、再统计。

下一章预告:第4章 函数与方法——从 C 的函数指针时代讲起,覆盖引用参数、默认参数、重载、const 成员函数、lambda 与函数对象,以及传值/传引用/传指针的三方对比。